

Inventory Management System

Week 1 Task — Project Planning and Architecture Design

Prepared as a Junior Java Developer Architecture Submission

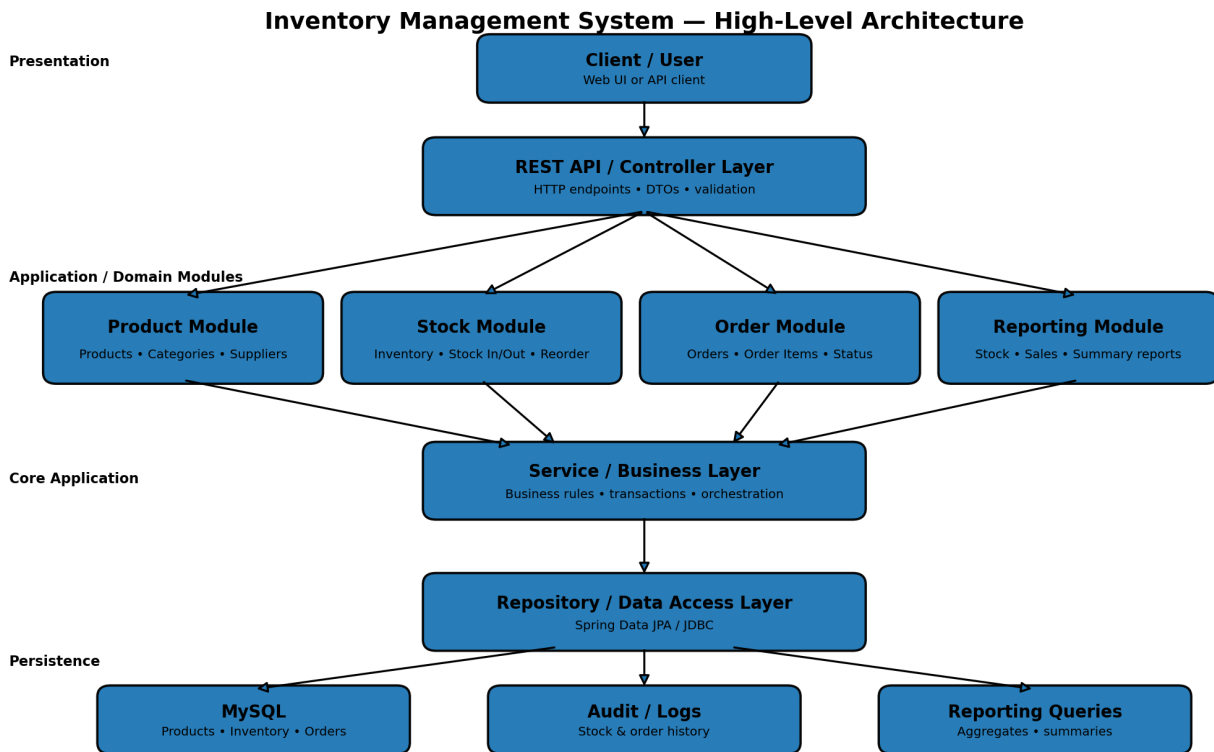


Figure 1: High-level architecture and module interaction.

Requirements at a Glance

Area	Planned Capability
Products	CRUD, SKU validation, category/supplier association
Inventory	Stock-in, stock-out, adjustment, low-stock detection, audit
Orders	Create, confirm, cancel, order totals and stock validation
Reports	Stock value, low-stock, category and order summaries
Technology	Java, Spring Boot, REST API, MySQL, JPA
Architecture	Layered modular monolith with dependency injection

1. Project Overview

The proposed Inventory Management System (IMS) is a small-business Java application intended for a retail store that needs a reliable way to manage products, stock levels, suppliers, customer orders, and operational reports. The purpose of this Week 1 design is not to implement the complete application, but to establish a practical architecture and a clear implementation plan that can be followed during development.

The system is designed around a layered architecture using Java and Spring Boot. A REST API will expose application capabilities to a web or mobile client, controller classes will handle HTTP requests, service classes will contain business rules, and repositories will manage persistence. MySQL is proposed as the relational database because inventory information is highly structured and requires consistency between products, stock transactions, and orders.

The design intentionally separates responsibilities. Product management should not directly manipulate database tables from a controller. Similarly, an order operation should not contain low-level SQL or HTTP concerns. This separation makes the application easier to test, maintain, debug, and extend.

2. Objectives

The main objectives are:

- Maintain accurate product and stock information.
- Allow authorized users to create, update, view, and deactivate products.
- Track stock additions, reductions, and adjustments.
- Prevent an order from reducing inventory below the available quantity.
- Maintain a history of important inventory transactions.
- Support basic order processing and order-status tracking.
- Provide useful reports such as current stock, low-stock products, stock value, and order summaries.
- Provide a clean REST API that can later be consumed by a React or other frontend.
- Keep the architecture modular enough to support future features such as authentication, notifications, barcode scanning, and cloud deployment.

3. Functional Requirements

3.1 Product Management The system shall allow users to add products with fields such as product ID, name, SKU, category, supplier, selling price, cost price, and reorder level. Products can be updated and deactivated rather than physically deleted when historical records depend on them.

3.2 Category and Supplier Management Products may belong to a category and may be associated with a supplier. The system should maintain these relationships so that users can filter products and generate supplier/category reports.

3.3 Stock Control The system shall maintain the current quantity for every active product. Stock can increase through purchases or stock adjustments and decrease through completed orders or approved adjustments. Every change should create an inventory transaction record.

3.4 Order Processing A user can create an order containing one or more order items. Before an order is confirmed, the system checks stock availability. On successful confirmation, the relevant quantities are reduced. Orders should have statuses such as CREATED, CONFIRMED, CANCELLED, and COMPLETED.

3.5 Reporting The system should provide reports for current inventory, low-stock products, total inventory value, stock movement, order totals, and category-level summaries.

3.6 Validation and Error Handling Required fields, positive quantities, valid prices, unique SKUs, and sufficient stock must be validated. The API should return meaningful HTTP status codes and structured error messages.

4. Non-Functional Requirements

The system should be maintainable, reliable, secure, testable, and reasonably scalable for a small retail business.

Maintainability: business logic must remain inside services rather than controllers or repositories.

Reliability: stock-changing operations must be transactional so that an order and its inventory updates do not leave the database in a partially updated state.

Performance: database queries should be indexed for fields such as SKU and product ID. Reporting queries should select only the required data.

Security: authentication and role-based authorization can be added as a separate security module. Passwords must never be stored in plain text.

Testability: services should depend on interfaces where appropriate, allowing unit tests to mock repositories.

Scalability: the initial system can be a modular monolith. If business growth later requires independent deployment, modules can be extracted into services with minimal changes to their internal responsibilities.

5. Proposed Architecture

The recommended architecture is a layered modular monolith.

Presentation Layer: REST controllers receive HTTP requests, validate input DTOs, call services, and return responses. Controllers should not contain business rules.

Service Layer: service classes coordinate application use cases. For example, `OrderService` validates an order, checks inventory, creates order records, and requests a stock reduction.

Repository Layer: repositories provide database access. Spring Data JPA can be used for normal CRUD and derived queries, while JPQL/native SQL can be used for specialized reporting.

Database Layer: MySQL stores normalized transactional data. Foreign keys maintain relationships between products, categories, suppliers, orders, order items, and inventory transactions.

The architecture is intentionally a modular monolith rather than microservices. For a small retail application, microservices would add operational complexity without providing enough immediate benefit. A well-structured monolith is easier to develop, deploy, monitor, and test while still allowing future extraction of modules.

6. Module Design

Product Module Responsibilities include product CRUD operations, SKU validation, category association, supplier association, price validation, and product activation/deactivation.

Stock Module Responsibilities include maintaining current quantity, processing stock-in and stock-out operations, checking reorder levels, and creating inventory transaction history. Stock modifications should occur through service methods rather than direct database updates from controllers.

Order Module Responsibilities include creating orders, validating order items, calculating totals, checking stock, changing order status, and coordinating inventory changes. Order confirmation should be atomic.

Reporting Module Responsibilities include read-only queries for inventory value, low-stock products, product movement, category summaries, and order summaries. Reports should not modify transactional data.

Supplier/Category Module This module manages reusable reference data and prevents inconsistent product classifications.

Audit Module Important operations such as stock adjustments and order status changes should be recorded with timestamp, operation type, product, quantity, and reference information. This improves traceability and debugging.

7. Proposed Database Design

The main tables are:

Product: id, sku, name, category_id, supplier_id, cost_price, selling_price, reorder_level, active, created_at.

Category: id, name, description.

Supplier: id, name, contact_name, email, phone.

Inventory: id, product_id, quantity, updated_at.

InventoryTransaction: id, product_id, transaction_type, quantity, reference_id, created_at.

CustomerOrder: id, order_number, status, total_amount, created_at.

OrderItem: id, order_id, product_id, quantity, unit_price, line_total.

Relationships are primarily one-to-many. A category can contain many products. A supplier can supply many products. An order has many order items. A product can have many inventory transactions. Inventory should have one current record per product.

The database should use primary keys, foreign keys, unique constraints on SKU and order number, and indexes on commonly searched columns. Monetary values should use DECIMAL rather than floating-point types.

8. Design Patterns and Principles

Layered Architecture Pattern The application is divided into controller, service, and repository responsibilities. This is appropriate because each layer has a clear reason to change.

Repository Pattern Repositories abstract persistence operations from business logic. The service layer can ask for a ProductRepository operation without knowing how the SQL is executed.

Service Layer Pattern Business use cases are centralized in service classes. For example, confirmOrder() can coordinate validation, stock checking, order persistence, and inventory transactions.

Strategy Pattern The system can use Strategy for report generation or stock adjustment rules. For example, different report strategies could generate low-stock, stock-value, or movement reports without placing every algorithm in one large class.

Factory Pattern A small factory can create inventory transaction objects based on transaction type such as STOCK_IN, STOCK_OUT, RETURN, or ADJUSTMENT. This becomes useful when transaction creation rules grow.

DTO Pattern Request and response DTOs prevent database entities from being exposed directly through the API. This also allows the API representation to evolve independently from the persistence model.

Dependency Injection Spring's dependency injection supplies services with repository dependencies. This reduces tight coupling and makes unit testing easier.

SOLID principles will guide implementation. Single Responsibility keeps modules focused; Dependency Inversion allows services to depend on abstractions; Open/Closed supports adding new report or transaction strategies without rewriting existing logic.

9. Key Business Flows

Stock-In Flow: 1. Client sends a stock-in request. 2. StockController validates the request DTO. 3. StockService verifies the product. 4. Inventory quantity is increased. 5. InventoryTransaction records the movement. 6. The transaction commits as one database operation.

Order Confirmation Flow: 1. Client submits an order. 2. OrderService validates the order items. 3. The service checks that every requested quantity is available. 4. The order is created or updated to CONFIRMED. 5. Inventory quantities are reduced. 6. STOCK_OUT transactions are recorded. 7. The total amount is calculated. 8. The entire operation commits atomically.

If any required step fails, the transaction should roll back so that the database does not contain a confirmed order without the corresponding stock reduction.

10. REST API Plan

Representative endpoints include:

GET /api/products — list products GET /api/products/{id} — get one product POST /api/products — create a product PUT /api/products/{id} — update a product DELETE /api/products/{id} — deactivate a product

GET /api/inventory — view inventory POST /api/inventory/stock-in — add stock POST /api/inventory/adjust — adjust stock GET /api/inventory/low-stock — find low-stock products

POST /api/orders — create order GET /api/orders/{id} — get order POST /api/orders/{id}/confirm — confirm order POST /api/orders/{id}/cancel — cancel order

GET /api/reports/stock-value — inventory value GET /api/reports/low-stock — low-stock report GET /api/reports/category-summary — category summary

The API should use standard HTTP semantics. Successful creation can return 201 Created, successful retrieval 200 OK, invalid input 400 Bad Request, missing resources 404 Not Found, and conflicts such as duplicate SKU 409 Conflict.

11. Java Code Scaffolding

A possible package structure is:

```
com.example.inventory ■■■■ controller ■■■■ service ■ ■■■■ impl ■■■■ repository ■■■■ entity ■■■■ dto ■■■■  
mapper ■■■■ exception ■■■■ strategy ■■■■ config
```

Example service interaction:

```
public interface StockService { InventoryResponse stockIn(Long productId, int quantity); InventoryResponse  
adjust(Long productId, int quantity); boolean hasSufficientStock(Long productId, int quantity); }
```

```
@Service public class StockServiceImpl implements StockService {
```

```
private final InventoryRepository inventoryRepository; private final InventoryTransactionRepository  
transactionRepository;
```

```
public StockServiceImpl( InventoryRepository inventoryRepository, InventoryTransactionRepository  
transactionRepository) { this.inventoryRepository = inventoryRepository; this.transactionRepository =  
transactionRepository; }
```

```
@Transactional public InventoryResponse stockIn(Long productId, int quantity) { // validate product and quantity //  
increase inventory // create STOCK_IN transaction // return response return null; }
```

The order service would follow a similar structure:

```
@Transactional public OrderResponse confirmOrder(Long orderId) { // load order // validate status // check stock  
for every item // reduce stock // record STOCK_OUT transactions // change order status // return response }
```

The actual implementation would replace the pseudocode with repository calls, validation, DTO mapping, and domain-specific exceptions.

12. Exception Handling and Validation

A centralized exception handler using `@RestControllerAdvice` is recommended. Custom exceptions such as `ProductNotFoundException`, `InsufficientStockException`, `DuplicateSkuException`, and `InvalidOrderStateException` make business failures explicit.

Validation annotations such as `@NotBlank`, `@Positive`, `@PositiveOrZero`, `@Email`, and `@Size` can be used on request DTOs. The API should return a consistent error structure containing timestamp, status, error code, message, and request path.

This approach prevents repetitive try/catch blocks in every controller and provides a predictable experience for frontend clients.

13. Testing Strategy

Unit tests should focus on service-level business rules. Examples include verifying that stock-in increases quantity, insufficient stock prevents order confirmation, duplicate SKUs are rejected, and cancelled orders cannot be confirmed.

Repository integration tests should verify database mappings and important queries.

Controller tests should verify HTTP status codes, validation errors, and response structures.

A complete testing plan should include normal cases, boundary values such as zero and negative quantities, missing products, duplicate SKUs, concurrent stock changes, and order cancellation scenarios.

For concurrency-sensitive inventory operations, the implementation should consider database locking or an appropriate optimistic/pessimistic concurrency strategy so that two simultaneous orders cannot incorrectly consume the same stock.

14. Security and Future Enhancements

Authentication and authorization should be introduced before production use. Roles such as ADMIN, INVENTORY_MANAGER, and SALES_USER can restrict operations. For example, only inventory managers may perform stock adjustments while sales users may create orders.

Future versions may include barcode scanning, supplier purchase orders, email alerts for low stock, dashboards, CSV/PDF reports, cloud deployment, caching, and audit dashboards.

If the business grows significantly, the reporting workload could eventually be separated from transactional operations. However, the first version should remain a modular monolith because it minimizes infrastructure complexity and is appropriate for the expected scale.

15. Architecture Decision Summary

The design chooses Java with Spring Boot, REST APIs, MySQL, and a layered modular-monolith architecture because these technologies are practical for a small retail application and align well with maintainable Java backend development.

The most important architectural decision is to keep business rules inside services and treat stock-changing operations as transactional use cases. Inventory is a consistency-sensitive part of the system: an order should never be confirmed while its stock reduction fails.

DTOs, repositories, dependency injection, centralized exception handling, and focused modules reduce coupling. Design patterns are used where they provide a real benefit rather than being added only for terminology. This makes the proposed solution feasible for a junior developer to implement while still demonstrating professional software engineering practices.

16. Conclusion

The proposed Inventory Management System provides a practical foundation for a small retail business. The architecture separates presentation, business logic, persistence, and reporting responsibilities while keeping the deployment model simple.

The design covers product management, stock control, order processing, reporting, validation, auditability, testing, and future security. The selected patterns—Layered Architecture, Repository, Service Layer, DTO, Strategy, Factory, and Dependency Injection—support maintainability and extensibility.

This Week 1 architecture can be directly converted into a Spring Boot implementation in later development phases. The next logical steps are database schema creation, Spring Boot project initialization, entity and repository implementation, service-level business rules, REST controllers, automated tests, and finally frontend integration.

Appendix A — Suggested Implementation Sequence

1. Initialize Spring Boot project and configure MySQL.
2. Create entities, relationships, constraints, and repositories.
3. Implement product, category, and supplier modules.
4. Implement inventory operations and inventory transaction history.
5. Implement order creation and transactional confirmation.
6. Add validation, custom exceptions, and global error handling.
7. Implement reporting queries.
8. Add unit, repository, and controller tests.
9. Add authentication and role-based authorization.
10. Integrate a frontend and prepare deployment.

Appendix B — Key Design Principle

The system should treat inventory quantity as business-critical state. Every operation that changes stock must be performed through a controlled service method and recorded as a transaction. This principle protects data consistency and provides an auditable history of stock movement.